

Dishcovery

A Matchmaking Software for Restaurants

Team presentation deck • replace any placeholder sections with your final report values

Team Members

Ryan Larrison • Yisilamujiang Muhetaer • Meghana Pula • Shazil Bajwa

Project Objective

What problem Dishcovery is designed to solve

Goal

Help users find restaurants that match their personal dining preferences instead of manually scrolling through many search results.

MAIN IDEA

Preferences + restaurant data

Ranked restaurant matches

Simple search and filtering
Restaurant profile details

What makes it useful?

Saves time choosing food

Personalized recommendations

Keeps favorites/preferences
Supports future restaurant updates

Cost Estimation

Use ONE method: Function Point or Application Composition

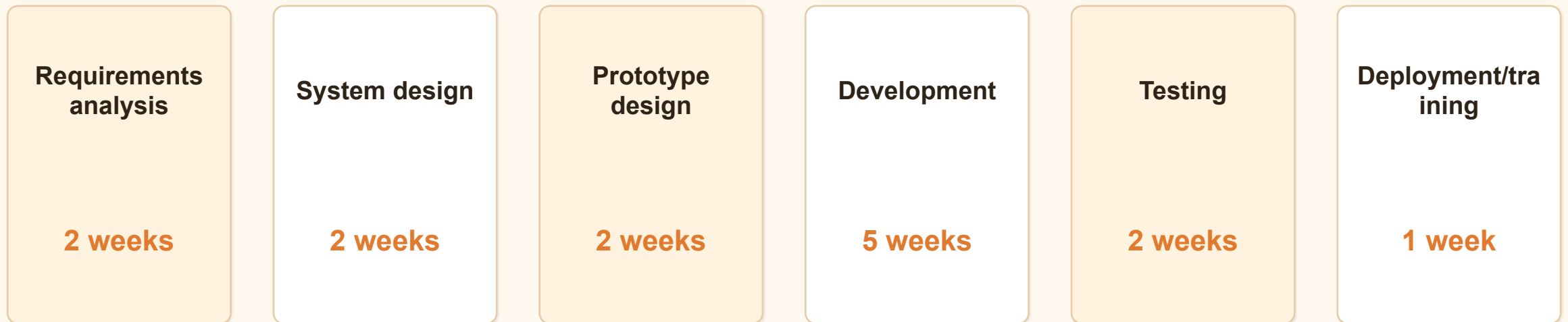
Function category	Count	Complexity	Weight	Count × Weight
User inputs	10	Average	4	40
User outputs	9	Average	5	45
User queries	6	Average	4	24
Data files/tables	4	Average	10	40
External interfaces	0	Average	7	

Hardware product	Quantity	Estimated cost per item	Total
Development laptops	4	\$900	\$3,600
Mobile device for responsive testing	1	\$300	\$300
External backup drive	1	\$100	\$100
Networking and miscellaneous equipment	1	\$100	\$100
Estimated hardware total			\$4,100

Total Function Points: 149
Hardware Cost: \$4,100
Software/Hosting Cost: \$320

Project Timeline

Add/verify dates from your final Deliverable #2 report



To finalize: add start date, end date, whether weekends count, and working hours per day.

Functional Requirements

Representative functions from Deliverable #1

User Registration & Login

Restaurant Information

Restaurant Search

Favorites Management

Restaurant Filtering

Preference Management

Restaurant Recommendation

These functions allow the user to search, personalize, and manage restaurant recommendations inside Dishcovery.

Non-Functional Requirements

Quality expectations for the system

Performance

Search results within 3 seconds

Usability

Simple interface; no training needed

Dependability

99% availability in normal operation

Security

Encrypted passwords + authentication

Compatibility

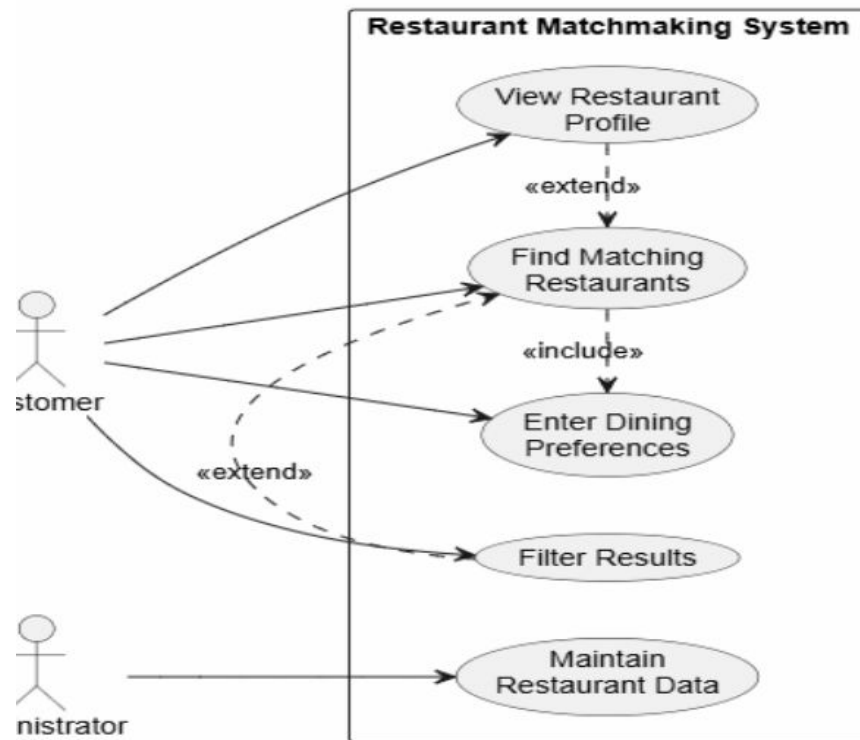
Modern browsers and mobile devices

Development

MVC architecture + GitHub version control

Use Case Diagram

Main actors and high-level interactions



Actors

Customer
Administrator

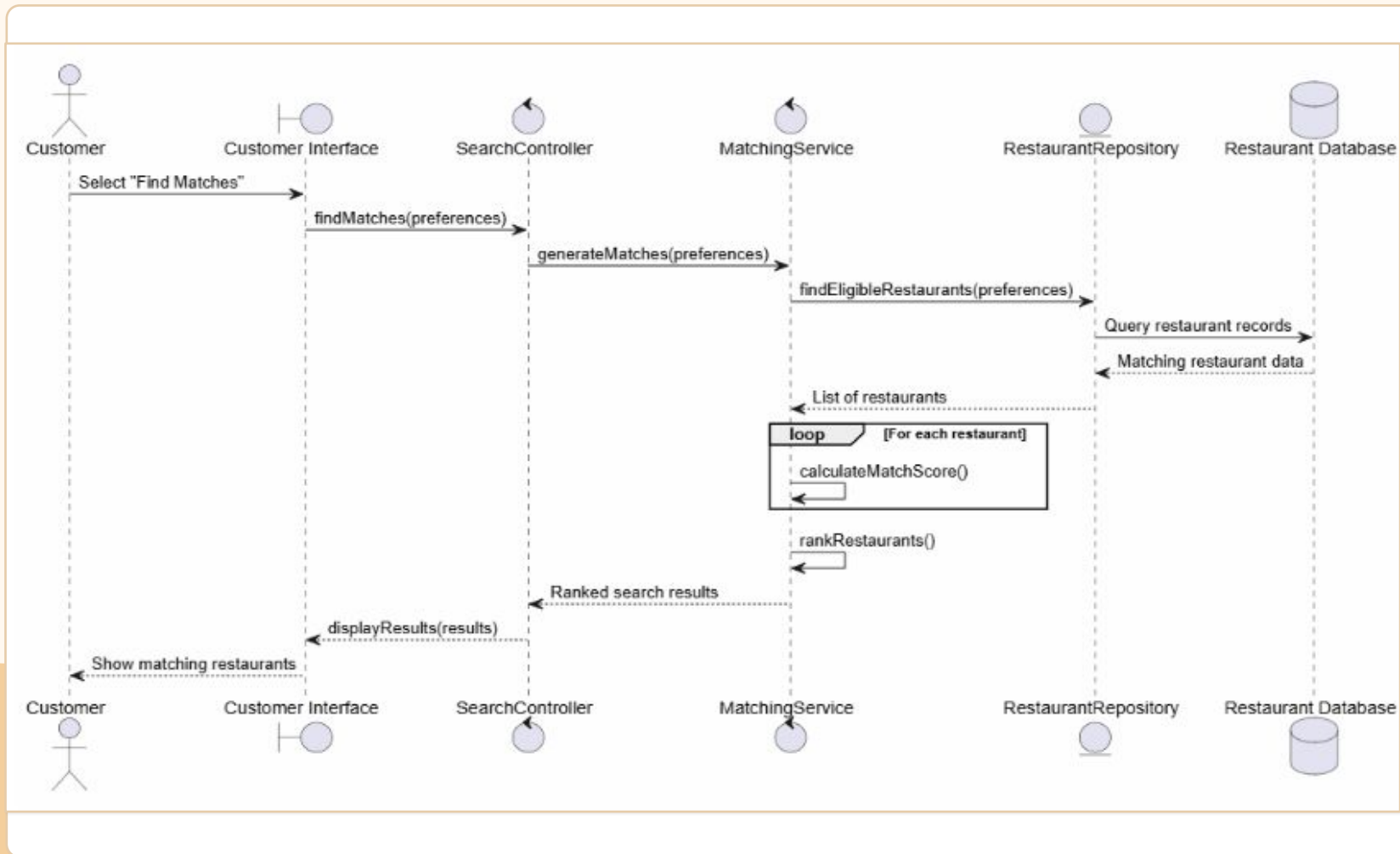
Main Use Cases

Enter dining preferences
Find matching restaurants
Filter results
View restaurant profile
Maintain restaurant data

Shows how users and admins interact with Dishcovery.

Sequence Diagram

Representative operation: matching restaurants

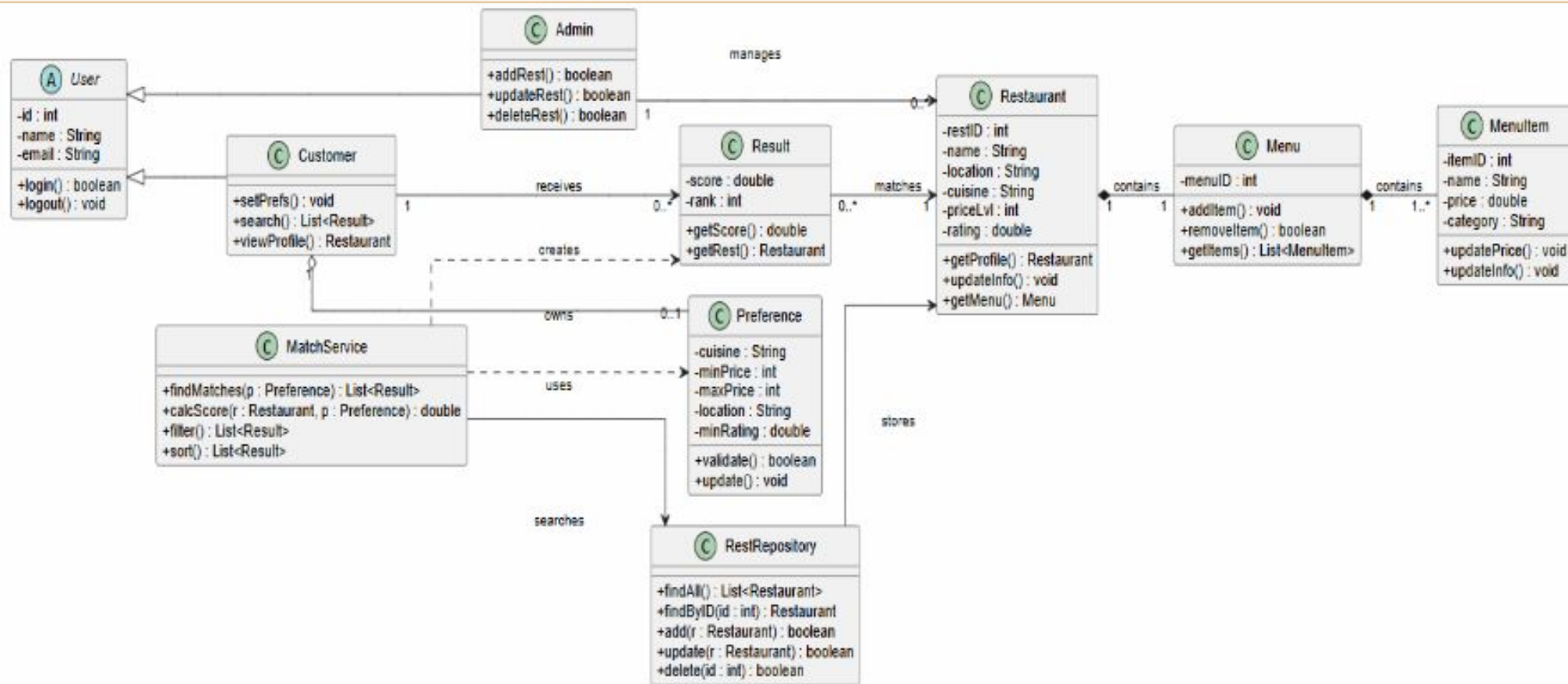


Matching Restaurants

Customer selects Find Matches
Controller sends preferences to matching service
Restaurant data is retrieved
Scores are calculated and ranked
Results are displayed

Class Diagram

Main objects and relationships in Dishcovery

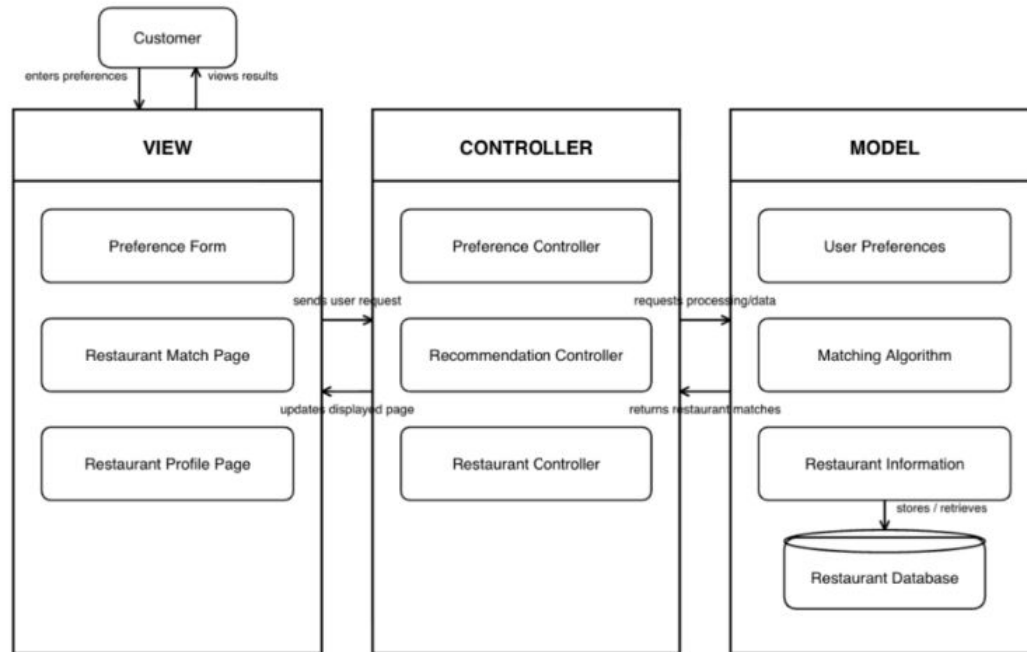


Key classes include User, Customer, Admin, Restaurant, Menu, MenuItem, Preference, Result, MatchService, and RestRepository.

Architectural Design

Model-View-Controller pattern

Dishcovery MVC Architectural Design



MVC

View: preference form, match page, profile page

Controller: preference, recommendation, restaurant controllers

Model: preferences, matching algorithm, restaurant database

Chosen because it separates UI, logic, and data

Comparison With Similar Designs

Replace or verify with citations in final report

Feature	Dishcovery	Yelp	Google Maps
Restaurant search	Yes	Yes	Yes
Preference-based matching	Main focus	Limited	Limited
Filtering	Yes	Yes	Yes
Favorites / saved places	Yes	Yes	Yes
Simple matchmaking flow	Yes	No	No

Main difference: Dishcovery focuses on matching users with restaurants based on personal preferences, not only general search results.

Test Plan and Results

Add your Java/JUnit screenshots and final outcomes

Unit Under Test

Example:
calculateMatchScore()
or filterByCuisine()

screenshot(next slide).

True Case

Input: matching
preference
Expected: true / high
score
Actual: true / high score
Status: Passed

False Case

Input: non-matching
preference
Expected: false / low
score
Actual: false / low score
Status: Passed

Note: “true and false test-case execution” can mean testing both logical outcomes, not leaving your JUnit suite broken.

collected 2 items

tests/test_matching_service.py::test_restaurant_matches_all_preferences PASSED

[50%]

tests/test_matching_service.py::test_restaurant_does_not_match_preferences FAILED

[100%]

===== FAILURES =====

test_restaurant_does_not_match_preferences

```
def test_restaurant_does_not_match_preferences():
```

```
    restaurant = FakeRestaurant(
```

```
        cuisine="Italian",
```

```
        location="Austin",
```

```
        price_level=4,
```

```
        rating=3.0,
```

```
        dietary_options="None"
```

```
    )
```

```
    preferences = FakePreferences(
```

```
        cuisine="Korean",
```

```
        location="Dallas",
```

```
        max_price_level=2,
```

```
        minimum_rating=4.0,
```

```
        dietary_options="Vegetarian"
```

```
    )
```

```
    result = MatchingService.score(restaurant, preferences)
```

```
    assert result.score == 100
```

```
    assert 3.0 == 100
```

```
    + where 3.0 = MatchResult(restaurant=<test_matching_service.FakeRestaurant object at 0x000002646D915590>, score=3.0, reasons=[]).score
```

tests\test_matching_service.py:76: AssertionError

===== short test summary info =====

FAILED tests/test_matching_service.py::test_restaurant_does_not_match_preferences - assert 3.0 == 100

===== 1 failed, 1 passed in 0.41s =====

Challenges, Lessons Learned, and Conclusion

Final evaluation of the project

Challenges

Coordinating four members
Keeping diagrams consistent
Estimating cost and timeline
Creating useful tests

Lessons Learned

Task delegation matters
Requirements guide design
Diagrams clarify the system
Testing should start early

Conclusion

Dishcovery demonstrates requirements analysis, system design, architecture planning, testing, and prototyping for a restaurant matchmaking application.